

Занятие 1. Агент как исполнитель: настройка окружения и проверка результата

Учебный материал по записи занятия от 2026-08-11 (73 минуты). Всё, что здесь описано, было показано вживую на одной задаче — игре 2048. Инструменты в демонстрации: Claude Code v2.1.227, модель Opus 5 (1М контекста, усилие medium), Codex CLI, скиллы frontend-design и superpowers, набор MCP-серверов. **Комплект занятия:** getting-started.md — с чего начать, если агентами ещё не пользовались · lesson-01.md — этот файл, конспект занятия · transcript.md — расшифровка записи с таймкодами · reference.md — справочник: каждый блок подробнее, со 130+ проверенными ссылками · lesson-01-subtitled.mp4 — запись с субтитрами · subtitles.srt — субтитры отдельно.

1. Главная мысль занятия

Написание кода стало дешёвым. Дорогой стала **проверка**: основное время разработчика и тестировщика уходит теперь на валидацию того, что сделал агент.

Отсюда следует практический вывод, вокруг которого построено всё занятие:

Работа с агентом — это не подбор формулировок в промпте, а **настройка окружения, в котором он работает, и организация проверки его результата.**

Один и тот же промпт в ненастроенной папке и в настроенном репозитории даёт разный результат. Разница не в словах, а в том, что лежит рядом: правила проекта, разрешения, подключённые инструменты, скиллы и проверочный контур.

2. Ландшафт инструментов

Все современные модели работают немного по-разному, но делают примерно одно и то же. На занятии всё показывалось на Claude Code и Opus 5, но выбор шире:

ИНСТРУМЕНТ	ГДЕ ЖИВЁТ	ЗАМЕЧАНИЕ ИЗ ЭФИРА
Claude (десктоп)	Приложение	Можно открывать папки, делать примерно то же, что в CLI
Claude Code	Терминал	Основной инструмент занятия; субъективно удобнее всего
Codex CLI	Терминал	Тулза от ChatGPT; используется как независимый аудитор
Китайские модели	Разное	«Тоже очень неплохие»
VS Code + плагин	Редактор	Можно поставить и Claude Code, и Codex

Чем агент отличается от чата в браузере: не нужно копировать код туда-сюда. Вы запускаете агента у себя на компьютере, и он работает с файлами напрямую.

Важное следствие: агент запускается в папке проекта и видит только её как проект. Соседняя папка для него — чужая территория (хотя доступ к машине у него есть). Поэтому в разных проектах один и тот же Claude Code ведёт себя по-разному — он настроен по-разному.

3. Четыре опоры доверия

Доверие к агенту не даётся авансом — оно набирается временем использования конкретной модели и складывается из четырёх вещей:

1. **Предсказуемость** — вы примерно понимаете, как модель себя ведёт.
2. **Ограниченность** — вы расставили ограничения, чтобы она не делала лишнего.
3. **Компетентность** — вы дали ей инструменты и знания, которых в ней нет.
4. **Проверяемость** — вы проверяете результат, а не верите отчёту.

Каждый следующий раздел — это способ добрать одну из этих опор: **CLAUDE.md** даёт ограниченность, MCP и скиллы — компетентность, хуки и независимый аудит — проверяемость.

4. Три прогона на одной задаче

Занятие построено как три параллельных сессии на одной задаче — игра 2048. Стоит воспроизвести их самостоятельно: это самый быстрый способ увидеть разницу.

T1 — голый промпт

напиши игру 2048

Результат: рабочий самодостаточный HTML-файл (~460 строк), открывается двойным кликом. Игра работает, выглядит прилично.

Что не так: мы в этом не участвовали. Модель решила всё сама — стек, архитектуру, объём. Мы не знаем, что под капотом, сколько это ест памяти и ресурсов. Доверия к результату практически нет.

T1b — промпт, сгенерированный моделью

Промежуточный приём: попросить ChatGPT или Codex составить промпт.

сделай промпт для реализации игры 2048

На выходе — структурированное задание: игровая логика, технические требования, ожидаемый результат. Результат по нему заметно лучше однострочника.

Но: требования тоже сгенерированы моделью. Это по-прежнему не наши требования — мы просто переложили придумывание с одной модели на другую.

Отдельно из эфира: писать супер-профессиональные промпты, как в 2024–2025 годах, больше не нужно — модели поумнели. Инвестировать стоит не в формулировку, а в окружение.

T2 — репозиторий с правилами

Новая папка, новая сессия. Первым делом создаётся `CLAUDE.md` — и дальше работа идёт по нему.

Что было показано на T2: - инициализация `CLAUDE.md` из ранее полученных требований; - ручная правка файла: условие победы 2048 → **64**, React/TypeScript → чистый HTML; - добавление запрета на удаление папки `Temp`; - проверка того, что агент подхватывает `CLAUDE.md` **без напоминания в промпте**; - инициализация git-репозитория и `.gitignore` руками агента; - переписывание внешнего вида скиллом `frontend-design`.

Итог: игра завершается на 64 — правило из `CLAUDE.md` соблюдено. Стек соответствует правилу. Внешний вид заметно лучше исходного.

T3 — Spec-Driven Development

Третья папка, скилл `superpowers`, полный цикл: brainstorming → спецификация → проверка спецификации независимой моделью → план → реализация.

Итог: спека в `docs/superpowers/specs/2026-08-11-game-2048-design.md`, 13 находок от Codex, чек-лист проверки вырос с 9 до 22 пунктов **до написания кода**.

5. Контекст

Что такое контекстное окно

Это объём данных, которым вы и модель можете обмениваться. Когда окно заполняется, модель не может нормально работать: контекст сжимается, часть данных теряется.

- У Claude — 1 миллион токенов. Для кодинга это существенно: помещается больше информации, и модель может писать больше кода, **потому что написанный код тоже идёт в этот же контекст**.
- У GPT-5 окно меньше — количество действий в одной сессии ограниченнее.

Что лежит в контексте

Команда `/context` показывает состав прямо сейчас:

- системный промпт;
- описания системных инструментов (bash, grep, работа с терминалом);
- файлы памяти — `CLAUDE.md` проекта и глобальные;
- скиллы;
- все ваши сообщения в диалоге.

Собственный промпт — лишь малая часть этого объёма. Отсюда правило: **думать надо о том, что попадает в контекст, а не о том, как красиво сформулировать запрос**.

Правила гигиены

ПРАВИЛО

ПОЧЕМУ

Новая задача — новая сессия

В заполненном контексте остаются старые баги и неактуальные данные; модель начнёт использовать их в новых ответах

Большую фичу делить на задачи по сессиям

Даже если это логическое продолжение

Следить за компактом

При переполнении происходит сжатие: автоматически берётся примерно 60–70 % последних ответов и 20–30 % первых; важная модель постарается подсветить сама, но получается не всегда

Компактить осознанно

`/compact` можно вызвать вручную и указать, что оставить

Держать хуки, скиллы и MCP в разумном объёме

Всё это занимает контекст ещё до первого вашего сообщения

Приём обхода: субагент — это независимая сессия внутри текущей, со своим контекстом. Он не примешивает свои поиски по файлам и мусорные шаги в основную сессию: наверх возвращается только сводка. Так решается задача «нужно много прочитать, но не засорять основную сессию».

Отдельно: если вы отошли от монитора и не заметили автоматического компакта, модель может начать работать хуже — в контекст, по её мнению, попадут не самые нужные файлы.

6. CLAUDE.md / AGENTS.md — первый круг доверия

Файл с информацией о проекте: требования, соглашения, разрешения, ограничения. Это **вектор для модели**: куда двигаться при разработке, что можно и чего нельзя.

- `CLAUDE.md` — для Claude Code.
- `AGENTS.md` — более универсальный вариант, понимается Codex и другими агентами.

Ключевые факты из демонстрации

Агент сам этот файл не создаёт. В прогоне T1 модель написала целое React-приложение и не создала ни `CLAUDE.md`, ни `AGENTS.md`. Без него агент в свободном полёте — он делает что хочет в рамках промпта.

Файл подхватывается по умолчанию. Указывать «используй `CLAUDE.md`» в промпте не нужно — это было проверено на T2 отдельно.

Файл нужно валидировать. Сгенерированный агентом `CLAUDE.md` содержит его собственные допущения — архитектуру, стек, критерии. Проверять надо двумя способами: - **глазами** — **обязательно**; - моделью: попросить пересмотреть файл и подсветить узкие места.

Это основной файл, по которому агент принимает решения. Ошибка в нём тиражируется на все последующие сессии.

Файл можно и нужно править руками. В эфире прямо в процессе менялось условие победы (2048 → 64) и стек (React/TS → чистый HTML), и агент это подхватывал.

⚠️ **Но не во время работы сессии.** Правка `CLAUDE.md` в момент, когда агент уже пишет код, привела к остановке работы и вопросам от модели. Новые правила подтягиваются со следующей сессии, а не в текущей.

Как держать файл в порядке

`CLAUDE.md` может быть огромным, но это не нужно и вредно:

- держать корневой файл **чистым и читаемым глазами** — чтобы он не запутывал самого агента;
- правила по отдельным фичам и экранам выносить в **отдельные файлы**;
- в `CLAUDE.md` оставлять **ссылки** на них — агент прочитает их автоматически.

Формулировки-запреты

Показательный эпизод. В файле было написано «не удалять Temp» — агент **удалил папку и при этом сообщил, что удалять нельзя**. Причина двойная:

1. Стоял режим **bypass permissions** — агент делает всё, что считает нужным.
2. Формулировка была мягкой.

После замены на «**ни при каких обстоятельствах** не удалять папку Temp» агент отказался выполнять команду и предложил либо подтвердить, что запрет устарел, либо оставить как есть.

Вывод: категоричность формулировки в правилах имеет значение, а режим разрешений — ещё большее.

7. Режимы разрешений

Это не переключатель «да/нет», а спектр. Из показанного в эфире:

РЕЖИМ	ПОВЕДЕНИЕ	КОГДА УМЕСТЕН
Ручной	Одобрям вообще всё	Незнакомый проект, первые дни
Одобрение правок	Спрашивает при изменении существующих файлов	Осторожная работа в живом коде
Планирование	Исследует и предлагает план, не правит	Разбор задачи перед реализацией
Auto	Решает сам: важное спрашивает, простое (например, безобидная команда в терминале без выхода в интернет) — нет	Рабочий режим по умолчанию
Bypass permissions	Не спрашивает ничего	Только когда вы понимаете риск — именно в этом режиме была удалена запрещённая папка

Переключение — `shift+tab` (в интерфейсе Claude Code режим отображается внизу экрана).

8. Три кита: MCP, скиллы, хуки

MCP — Model Context Protocol

Интерфейс подключения агента к внешнему инструменту. Грубо говоря, протокол, через который агент дёргает ручки в приложении: браузер, Figma, Photoshop, 3ds Max — что угодно, для чего написан MCP-сервер.

MCP даёт агенту то, чего у LLM нет в принципе: открыть браузер, походить по страницам, заполнить формы.

Показанные в `/mcp` серверы:

СЕРВЕР	ЧТО ДАЁТ
Playwright	Управление браузером. Основной инструмент тестировщиков и веб-разработчиков
Context7	Актуальная документация практически по всем фреймворкам и языкам. Нужен потому, что модель обучена в прошлом и её память по библиотекам устаревает
Stitch	Дизайнерский инструмент от Google
Pencil	Аналог Figma, позволяет агенту делать дизайн

Правило выбора: подключать те MCP, которые реально нужны в этом проекте, — каждый занимает контекст.

Скиллы — компетенция агента

Скилл — это набор правил в MD-файле, описывающий поведение модели в определённой ситуации. Модель их знает, использует, и её поведение оказывается в заданных рамках. Современные «агенты» — по сути тоже скиллы.

Где брать: - готовые с GitHub — ориентироваться на количество звёзд; - `frontend-design` от Anthropic — один из самых крупных, показан в эфире; - писать свои — можно просто попросить Claude их сделать.

Как ставить: попросить агента установить скилл и дать ссылку на репозиторий. В эфире установка заняла считанные секунды.

Эффект, увиденный вживую: без скилла модель сама выбрала оформление и получила бледную страницу. Со скиллом `frontend-design` — сама же выбрала, но подключила дополнительные библиотеки, украшения и анимации. Результат стал существенно лучше при одинаковой задаче.

Свой скилл — главная долгая инвестиция. Пример из эфира: LLM неплохо разбирается в Android, но не отлично. Вы пишете свой скилл под Android, куда складываете накопленный опыт — как тестировать, что агент должен уметь, какой стек использовать, — и дальше вся разработка идёт через него.

Всегда лучше немного потратить время, найти хороший скилл по своему стеку и доработать своими знаниями. В долгую это качественнее и предсказуемее.

Стоимость в контексте: скиллы, загружаемые по запросу, можно держать в неограниченном количестве — они просто лежат на диске. Скиллы, загружаемые сразу (например, `superpowers`, вызываемый по хуку), едят контекст постоянно — на хороших моделях не критично, но помнить об этом надо.

Хуки — автоматизация поведения

Хук — заданное поведение в определённый момент работы. Не обязательный, но очень удобный инструмент.

Примеры из эфира: - **валидация плана сторонней моделью:** каждый созданный план обязан пройти проверку, агент не двинется дальше, пока хук не отработает; - **сборка билда в Xcode** → поднять симулятор и показать приложение; - **коммит по событию** — вместо надежды на то, что агент вспомнит сам.

Смысл хуков — не включаться в процесс руками там, где этого не требуется.

Слои настройки — порядок

Чтобы доверять работе агента в репозитории:

1. **CLAUDE.md** — правила проекта.
2. **MCP** — инструменты, которые реально нужны.
3. **Скиллы** — компетенция под ваш стек.
4. **Хуки** — автоматизация проверок и рутины.

9. Spec-Driven Development

Самый качественный на сегодня паттерн работы с ИИ. Модель через хуки заставляют работать по фиксированной последовательности:

спецификация → план → разработка → проверка независимым агентом

Инструмент: скилл `superpowers` — ставится с GitHub, дальше сам настраивает хуки, и разработка идёт через SDD.

Как это выглядело в T3

1. Промпт: «давай продумаем разработку и начнём игру 2048».
2. Хук автоматически запустил `superpowers:brainstorming`.
3. Агент задал уточняющие вопросы: объём фич (взяли `best score`), стиль, анимации.
4. Набросал архитектуру — разделение «модель / представление», три слоя в одном файле, чистые функции над матрицей 4×4.
5. Человек вмешался: «нужно растягивать и сжимать для мобилок» — и попросил писать спецификацию.
6. Спека записана в `docs/superpowers/specs/2026-08-11-game-2048-design.md`.
7. Спека проверена через Codex.
8. Только после подтверждения — переход к плану реализации.

Ключевой момент шага 5: на этапе спецификации дёшево отсечь всё лишнее, что модель придумала сама. Позже это будет стоить переписанного кода.

Альтернатива без скиллов: можно просто попросить агента задавать вам вопросы — использовать ИИ как интервьюера. Работает, но готовый инструмент удобнее.

Усилие модели (effort)

Чем выше усилие — тем дольше работает и тем лучше думает; чем ниже — тем быстрее. В эфире стоял medium ради скорости, и прозвучало прямо: на x-high модель, скорее всего, сама додумалась бы до растягивания на весь экран.

Но это не гарантия. Именно поэтому и нужны CLAUDE.md, правила, MCP и хуки — они не зависят от того, «догадается» ли модель.

10. Аудит независимой моделью

Второй агент проверяет работу первого. В эфире Claude Code сам вызывал Codex через терминал:

проверь через Codex

```
codex exec --skip-git-repo-check "Прочитай файл docs/superpowers/specs/2026-08-11-game-2048-  
design.md —  
это спецификация игры 2048 на чистых HTML/CSS/JS без сборки. Проведи ревью спеки:  
найди логические ошибки, противоречия, пробелы, неверные технические утверждения"
```

Зачем это нужно

ИИ пишет огромное количество кода, и отсмотреть его целиком не успеваешь — даже на небольшой фиче, если она задевает много файлов. Базово отсечь проблемы сторонней моделью — по факту стандарт рынка сегодня.

Что дал аудит в эфире

Проверка спецификации (кода ещё нет, поэтому быстро): **13 находок, все по делу, отклонять было нечего.** Среди них:

<code>container-type</code> и <code>gap: 2.2cqi</code> на одном узле	<code>cqi</code> считается от ближайшего предка-контейнера, а не от самого элемента — зазоры посчитались бы от вьюпорта
<code>min(92vmin, 480px)</code>	В альбомной ориентации 92 % высоты + шапка не влезали на экран
Ввод не блокировался при оверлее победы	<code>over === false</code> , и игрок продолжал двигать плитки под экраном
Победа и поражение одним ходом	Не был зафиксирован приоритет состояний
Перезапуск CSS-анимации	Опора на <code>animationend</code> не работает при двух быстрых ходах подряд
<code>maximum-scale=1</code>	Отключает pinch-zoom всей страницы — барьер для слабовидящих
<code>changedTouches</code> , <code>touchcancel</code> , <code>passive: false</code>	В <code>touchend</code> список <code>touches</code> уже пуст; без <code>passive: false</code> браузер игнорирует <code>preventDefault()</code>
Парсинг <code>best</code> из <code>localStorage</code>	<code>NaN</code> из испорченного значения отправил бы все сравнения счёта

Итог: чек-лист проверки вырос **с 9 до 22 пунктов** — и всё это до написания единой строки кода.

Проверка готового приложения (T2) заняла около 5–6 минут против ~2 минут на спецификацию. На большом объёме кода разница будет кратно больше.

Как усилить аудит

Codex вызывался просто с промптом. Если вызвать его **со скиллом по нужному стеку** (например, по HTML-разработке), он найдёт больше. Отсюда практика: держать набор агентов-специалистов по направлениям и вызывать нужного.

11. Экономика: куда вкладывать токены

Правило, сформулированное в эфире прямо:

Чем меньше сил вложено в начале, тем больше проблем будет в конце.

ЭТАП	МОДЕЛЬ И УСИЛИЕ	ПОЧЕМУ ТАК
Спецификация	Дорогая модель, максимальное усилие	Спека маленькая — по токенам почти бесплатно, а определяет всё
Проверка спеки	Другая модель / независимый агент	Тоже дёшево: проверять нечего, кроме текста
План	Дорогая модель	По хорошему плану будет написан хороший код
Написание кода	Можно экономить	Хороший код по хорошему плану пишут многие модели, не все из них дорогие

Продумать архитектуру, фичи, экраны — задача для умных моделей. Набить код по готовому плану — нет.

Оркестрация моделей работает по той же логике: дорогая и умная модель управляет более дешёвыми, создаёт их и раздаёт задачи. Она же выступает частичным тестером их результатов и агрегатором того, что они нашли, — то есть забирает на себя часть вашей работы.

Подписки

ПОДПИСКА	ИЗ ЭФИРА
Claude Pro	Лимит закончится быстро; обнуляется каждые 5 часов. В трёх пятичасовых окнах можно сделать что-то интересное
Claude Max (\$100)	Хватит на всё
Claude Max (\$200)	Автор ведёт ~10 проектов одновременно и не всегда упирается в лимиты
Codex (\$20)	Больше лимитов за чуть более слабый функционал; пользоваться можно. Claude сильнее в кодинге

Расход сильно зависит от режима работы: постоянная работа на максимальных усилиях и лишние действия сжигают лимит быстро.

12. Что LLM делает хорошо, а что плохо

Хорошо — всё, что сводится к терминальным командам:

- git: ветвление, мерж, разбор дерева, коммиты, worktrees, pull requests;
- DevOps-задачи;
- написание тестов на существующий код;
- инициализация репозитория, `.gitignore`.

Это первое, на что стоит перевести агента.

Требуется вашего участия:

- собственно разработка — нужно учитывать свой стек;

- **дизайн — самое сложное.** Его нужно отсматривать глазами, и усилий на него уходит больше всего.

При этом даже отличный результат (например, сгенерированный `.gitignore`) перепроверяется глазами — это заняло в эфире несколько секунд.

13. Работа в команде

- `CLAUDE.md` **ходит вместе с репозиторием** и настраивается всей командой.
 - Каждый добавляет свои правила **через pull request**, они согласовываются тимлидом или куратором проекта.
 - У крупных компаний (включая Anthropic) со временем накапливаются очень мощные `CLAUDE.md` и файлы агентов, которыми делятся в open source. Это отличный материал: взять, подправить под свой проект и получать хорошие результаты.
-

14. Чек-лист старта нового проекта

Порядок, собранный из эфира — в идеале это самое первое действие в новой папке:

1. Создать папку проекта, запустить агента **в ней**.
 2. Инициализировать `CLAUDE.md` (или `AGENTS.md`) и документацию.
 3. **Прочитать `CLAUDE.md` глазами**, вычистить придуманную моделью, вписать свои ограничения категоричными формулировками.
 4. Инициализировать git-репозиторий, проверить `.gitignore` глазами.
 5. Выбрать режим разрешений осознанно (не bypass, если в проекте есть что терять).
 6. Подключить нужные MCP (браузер, документация — по потребности).
 7. Поставить скиллы под свой стек; со временем — написать свой.
 8. Настроить хуки: валидация плана, коммиты, сборка.
 9. Работать через Spec-Driven Development: спека → проверка спеки → план → код.
 10. Аудит результата независимым агентом; правки — по находкам.
 11. Коммитить каждый значимый шаг.
-

15. Ошибки, показанные вживую

Их стоит воспроизвести — они учат лучше правил.

ЧТО СЛУЧИЛОСЬ	ПРИЧИНА	УРОК
Агент удалил запрещённую папку Temp и сам сообщил, что удалять её нельзя	Режим bypass permissions + мягкая формулировка запрета	Запрет словами работает только вместе с режимом разрешений
Правка CLAUDE.md во время работы сессии остановила работу и вызвала вопросы	Файл читается на старте сессии	Правила меняются между сессиями, а не внутри
Модель создала кучу файлов там, где просили один index.html	В CLAUDE.md осталось неубранное правило	Каждое правило в файле действует; мусор в файле — мусор в результате
Сгенерированный CLAUDE.md содержал чужую архитектуру и стек	Файл писала модель	Проверять глазами, править руками

16. Домашнее задание и что дальше

Формат ДЗ: по группам (разработка, тестирование). Задаётся **вектор** задания, конкретный проект выбираете сами — жёсткой привязки к одному типу проекта не будет.

Критерии оценки (полностью не раскрываются):

- насколько креативно и ново то, что вы использовали;
- насколько глубоко вы погрузились в использование разных инструментов;
- важен **не столько конечный результат, сколько вовлечённость**: сколько сил вложено, что и как докручено в репозитории.

Простой по функциональности сервис с богатой обвязкой инструментов ценится выше вылизанного, но собранного «в один промпт».

Темы следующих занятий:

- security и песочницы для агентов;
- оркестрация агентов;
- более глубокий harness вокруг агента;
- углублённый контекст-менеджмент и оптимизаторы контекста.

17. Куда дальше

Каждый блок этого конспекта разобран подробнее в [reference.md](#) — с первоисточниками, рабочими конфигами и командами:

Ссылки на всё, что показывалось	1. Репозитории и официальные ресурсы
Скиллы, <code>frontend-design</code> , свой скилл под стек	2. Скиллы
<code>superpowers</code> , спека → план → код → проверка	3. <code>superpowers</code> и Spec-Driven Development
MCP-серверы, Playwright, Context7	4. MCP
Хуки: события, протокол, готовые примеры	5. Хуки
Контекст, компакт, усилие, тарифы и лимиты	6. Контекст, модели и экономика
Режимы разрешений, песочницы, инциденты	7. Разрешения и безопасность

18. Глоссарий

ТЕРМИН	ЗНАЧЕНИЕ
Контекстное окно	Объём данных, которым обмениваются вы и модель. У Claude — 1M токенов
Компакт	Сжатие переполненного контекста. Автоматически берётся ~60–70 % последних ответов и 20–30 % первых
Сессия	Один чат с агентом со своим контекстом. Новая задача — новая сессия
Субагент	Независимая сессия внутри текущей; возвращает назад только сводку
CLAUDE.md / AGENTS.md	Файл правил проекта, читается агентом автоматически
MCP	Model Context Protocol — подключение агента к внешнему инструменту
Скилл	MD-файл с правилами поведения модели в определённой ситуации; компетенция агента
Хук	Заданное поведение в определённый момент работы агента
SDD	Spec-Driven Development: спека → план → разработка → независимая проверка
Effort	Усилие модели: выше — дольше и качественнее, ниже — быстрее
Bypass permissions	Режим, в котором агент не спрашивает разрешений вообще
Оркестрация	Умная модель управляет дешёвыми, раздаёт им задачи и агрегирует результаты